

Q1: Implement NOT, AND, OR Gates Using Only NAND Gates

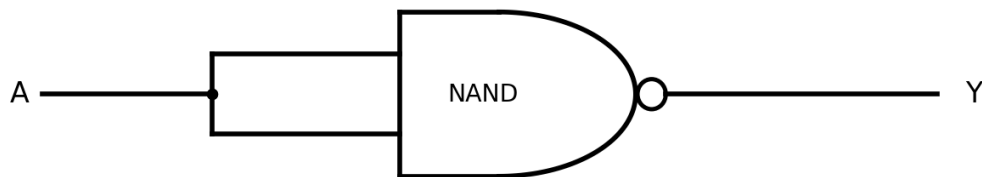
1. Aim

To implement NOT, AND, and OR logic gates using only NAND gates in HDL, draw their schematic diagrams, verify their Boolean expressions, and observe the simulation output using suitable test inputs.

2. Schematic Diagram

A. NOT Gate using NAND

Both inputs of the NAND gate are connected to the same input A.

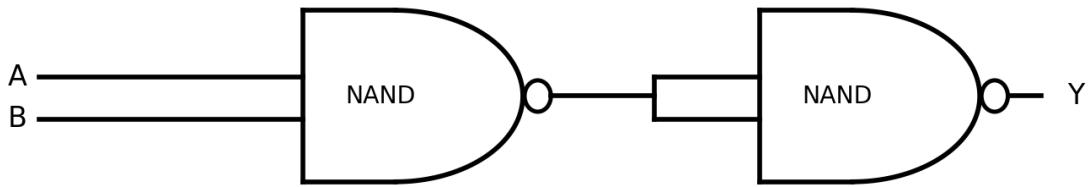


$$Y = (A \cdot A)' = A'$$

Boolean Expression: $Y = (A \cdot A)' = A'$

B. AND Gate using NAND

The first NAND produces the complement of $A \cdot B$. The second NAND has its two inputs tied together, so it acts as an inverter.

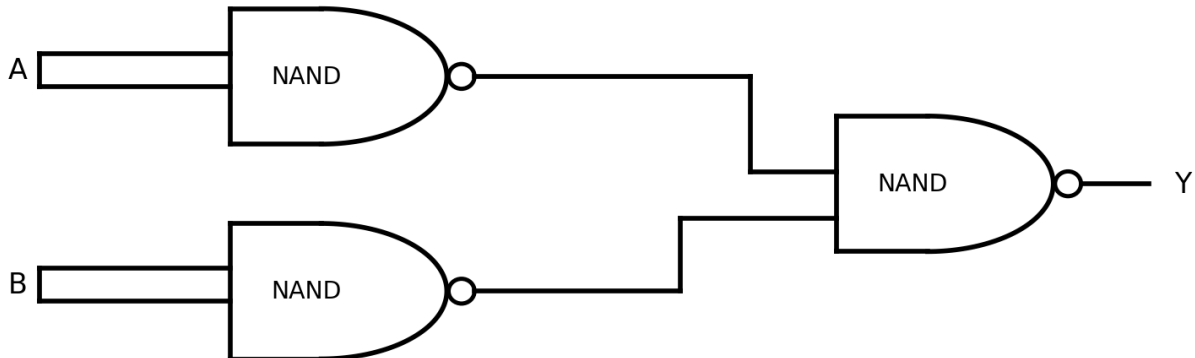


$$X = (A \cdot B)' \quad Y = (X \cdot X)' = A \cdot B$$

Boolean Expression: $X = (A \cdot B)'$; $Y = (X \cdot X)' = A \cdot B$

C. OR Gate using NAND

The first two NAND gates act as NOT gates for A and B. Their outputs are then connected to a third NAND gate.



$$A' = (A \cdot A)' \quad B' = (B \cdot B)' \quad Y = (A' \cdot B')' = A + B$$

Boolean Expression: $A' = (A \cdot A)'$; $B' = (B \cdot B)'$; $Y = (A' \cdot B')' = A + B$

3. HDL Code

```
// NOT Gate using NAND
module not_using_nand(input A, output Y);
    nand(Y, A, A);
endmodule

// AND Gate using NAND
module and_using_nand(input A, input B, output Y);
    wire X;
    nand(X, A, B);
    nand(Y, X, X);
endmodule
```

```

endmodule

// OR Gate using NAND
module or_using_nand(input A, input B, output Y);
    wire A_not, B_not;
    nand(A_not, A, A);
    nand(B_not, B, B);
    nand(Y, A_not, B_not);
endmodule

```

4. Testbench

```

module tb;
    reg A, B;
    wire NOT_Y, AND_Y, OR_Y;

    not_using_nand NOT1(.A(A), .Y(NOT_Y));
    and_using_nand AND1(.A(A), .B(B), .Y(AND_Y));
    or_using_nand OR1(.A(A), .B(B), .Y(OR_Y));

    initial begin
        $monitor("Time=%0t A=%b B=%b NOT=%b AND=%b OR=%b",
            $time, A, B, NOT_Y, AND_Y, OR_Y);

        A=0; B=0; #10;
        A=0; B=1; #10;
        A=1; B=0; #10;
        A=1; B=1; #10;

        $finish;
    end
endmodule

```

5. Simulation Output

The simulation should produce the following logical output for the four test cases:

Chip And	Eval	Reset	Clock: 0
Input pins			
a	0		
b	0		
Output pins			
out	0		
Internal pins			
x	1		
Fast			

Chip And		Eval	Reset	Clock: 0
Input pins				
a	1			
b	0			
Output pins				
out	0			
Internal pins				
x	1			

Chip And		Eval	Reset	Clock: 0
Input pins				
a	☐			
b	☐			
Output pins				
out	☐			
Internal pins				
x	☐			

Chip Or		Eval	Reset	Clock: 0
Input pins				
a	☐			
b	☐			
Output pins				
out	☐			
Internal pins				
x	☐			
y	☐			

Chip Or		Eval	Reset	Clock: 0
Input pins				
a	1			
b	0			
Output pins				
out	1			
Internal pins				
x	0			
y	1			

Chip Or		Eval	Reset	Clock: 0
Input pins				
a	1			
b	1			
Output pins				
out	1			
Internal pins				
x	0			
y	0			

Chip Not

Eval Reset Clock: 0

Input pins

☐ 0

Output pins

☒ 1

Chip Not

Eval Reset Clock: 0

Input pins

☒ 1

Output pins

☐ 0

6. Truth Table

A	B	NOT A	AND	OR
0	0	1	0	0
0	1	1	0	1
1	0	0	0	1
1	1	0	1	1

7. Result

The NOT, AND, and OR gates were successfully implemented using only NAND gates. The schematic diagrams, HDL implementation, Boolean expressions, and simulation results agree with the expected truth tables.